Introduction to Formal Mathematics with Lean 4

For newcomers to formalization methods

Xingyu Zhong

Beijing Institute of Technology

2025-09-17

Table of Contents

1. What is formalization
2. Why formalize
3. Why now formalize
4. How to formalize

Table of Contents

1. What is formalization
2. Why formalize
3. Why now formalize
4. How to formalize

Natural language vs. formal language

- ambiguity in natural language
  - implicit assumptions
  - skipping details: "It's clear that we have..."
  - "viewed as" arguments: $V^{**} = V$, $(A \times B) \times C = A \times (B \times C)$ [1]
  - abuses of notation: $3 \in \mathbb{Z}/5\mathbb{Z}$, $\mathbb{C} \subseteq \mathbb{C}[x]$
- precision in formal language
  - computer programs are formal languages

[1] Knowledgable audience may recognize them as examples of natural isomorphisms in category theory.

Mathematical proofs vs. Computer programs[2]

Logic	Programming
proposition	type
proof	term
proposition is true	type has a term
proposition is false	type doesn't have a term
logical constant `TRUE`	unit type
logical constant `FALSE`	empty type
implication $\to$	function type
conjunction $\wedge$	product type $\prod$
disjunction $\vee$	sum type $\sum$
universal quantification $\forall$	dependent product type $\prod$
existential quantification $\exists$	dependent sum type $\sum$

Table: Curry–Howard correspondence

[2] see also Computational Trilogy, with category theory as the third vertex

Set theory vs. Type theory

- Mathematicians choose axiomatic set theory (with first-order logic) as the foundation of mathematics.
  - naive set theory fits human's intuition well
- Type theory is an alternative foundation that is equally expressive, but more suitable for computer formalization.

Set Theory	Type Theory
everything is a set	everything has a type
$3 \in \mathbb{R}$ is a proposition	$(3 : \mathbb{R})$ is a typing judgment
$\mathbb{Q} \subseteq \mathbb{R}$ is an inclusion	$\mathbb{Q} \to \mathbb{R}$ is a type conversion

What is Lean 4

- A modern functional programming language designed for theorem proving

  "Lean is based on a version of dependent type theory known as the Calculus of Constructions, with a countable hierarchy of non-cumulative universes and inductive types." — *Theorem Proving in Lean 4*

Lean's dependent type theory

- Dependent type theory is a powerful extension of type theory where
  - types may depend on terms "given before" them
  - first-order logic can be implemented in dependent type theory
- functions, inductive types and quotient types[3] are the basic methods to construct new types.

Set Theory	Lean's dependent type theory
$\forall x \in \mathbb{R},\ x^2 \geq 0$	has type $(x : \mathbb{R}) \to (x^2 \geq 0)$
$(n \in \mathbb{N}) \mapsto (1, 0, \ldots, 0) \in \mathbb{R}^n$	has type $(n : \mathbb{N}) \to \mathbb{R}^n$
$\{0, 1\} = 2$ is a set equality	make no sense
cardinality is an equivalence class	is a quotient type
Russell's paradox	Girard's paradox

[3] Though seemingly redundant, there are reasons for making quotient types as a fundamental constructing method. funext thesis

An example Lean 4 code

- FLT
- TendsTo

Table of Contents

1. What is formalization
2. Why formalize
3. Why now formalize
4. How to formalize

The rise of AI

AI excels in Python. Why not Lean?

- Automated theorem proving
  - especially those "abstract nonsense"
  - full-auto (create a proof without human interaction)
  - semi-auto (suggest tactics)
    - `exact?`, Github Copilot, ...
- Natural language to formal language
  - automatically transplanting textbooks and papers into Lean
  - full-auto (translate without human interaction)
  - bolt-action (search for existing theorems)
    - LeanSearch, LeanExplore, ...
  - Converse? Already happening!
- Proposing conjectures
  - on which facts should we care about

Rigor matters

- It's the foundation of mathematics
- Imprecise natural language often leads to misunderstandings and glitches
  - Especially when proofs get longer and longer
- formalization fully confirms the correctness of a theorem
  - things that are too "technical" (boring) or simply impossible to verify by oneself
    - e.g. classification of finite simple groups
    - e.g. "technical"

"I spent much of 2019 obsessed with the proof of this theorem, almost getting crazy over it. In the end, we were able to get an argument pinned down on paper, but I think nobody else has dared to look at the details of this, and so I still have some small lingering doubts." — *Peter Scholze*

“Mathematical engineering”

- manipulating tons of theorems and proofs with mature software engineering techniques
- referencing existing theorems as dependencies
- collaborative work across the globe

The beauty of the system: you do not have to understand the whole proof of FLT in order to contribute. The blueprint breaks down the proof into many many small lemmas, and if you can formalise a proof of just one of those lemmas then I am eagerly awaiting your pull request. — *Kevin Buzzard on the FLT Project*

Formalization as learning

- proofs with infinite detail
  - intuitive textbooks, rigorous formalization
- makes us understand things better
  - Global: How to build natural numbers from scratch?
    - natural number game
    - A journey to the world of numbers, by Riccardo Brasca
  - Local: reducing the cognitive load
    - (With good organization at the beginning) you can focus on small parts of the proof at a time

Table of Contents

1. What is formalization
2. Why formalize
3. Why now formalize
4. How to formalize

Formalization becomes more accessible

Mathematician-friendly languages, interfaces, tools and community emerges:

- Lean 4 with VSCode extension, modern interactive theorem prover made for mathematicians
- formalization becomes more and more fashionable
- big names works on formalization:
  - Kevin Buzzard works on formalizing FLT
  - Peter Scholze's work on condensed mathematics has been formalized
  - Terrence Tao gave a talk on formalization in IMO 2024 and wrote a Lean 4 companion of his book "Analysis I" recently
- computer scientists and volunteering mathematicians run Lean 4 community collaboratively

Mathlib 4 is expanding explosively

By the time of 2025/09/16, Mathlib 4 has[4]

Lines of code	Definitions	Theorems	Contributors
1950000	115438	232204	653

- undergraduate may contribute: some low-hanging fruits

[4] Statistics fetched from Mathlib statistics

Table of Contents

1. What is formalization
2. Why formalize
3. Why now formalize
4. How to formalize

The goal

The goal, at the end of this course, is

- to get used to think formally
- to migrate from set theory to dependent type theory
- to practice basic skills to translate statements and proofs into Lean 4
- to know how to find existing theorems, how the community works
- to acquire enough common senses to read the bibles (MiL, TPiL, Mathlib 4 Doc, etc.) by yourself for future formalization projects
- (optimistically) to set up a Lean 4 formalization club at BIT!

How will we learn

As mathematicians, we learn Lean 4 to formalizing mathematics. We learn by practice.

- Dozens of Lean files packed with well-organized examples and exercises suffice to get you started, suitable for both guided study and self study. Most lectures will be given in this style.
- This style of teaching is inspired by Kevin Buzzard's 2024 course on formalising mathematics in the Lean theorem prover and many other courses.

What we won't cover

Due to the limited time, my personal inability and the design of this course, we might not be able to cover:

- a deep discussion into dependent type theory or Lean as a programming language itself
  - Read TPiL for a Lean 4 tutorial that emphasizes on the dependent type theory.
  - Read FPiL for a Lean 4 tutorial that focuses more on functional programming (monads, etc.)
  - Refer to Lean Language Manual for precise specifications.
- systematic exposition of how a particular branch of mathematics is formalized in Mathlib 4
  - Read MiL for this purpose.
- how to organize a massive formalization project from scratch, i.e. project management
  - somewhat subtle, might can only be learned by reading Mathlib codes and practical experience

Disclaimer

- Formalization is tedious in its nature, Lean is no exception
- Type conversions can be an extra burden (exclusive for type-theory-based systems)
- Knowledge needs to be re-learned before being referenced
- Different people may formalize the same thing in different ways

If these do not scare you away...

Welcome aboard. Have fun formalizing mathematics!

Resources

- course repository
- online documentation